VLSI Design Lab: Verilog HDL Reference Manual

COMPLETE VERILOG DESIGN CODES & TESTBENCHES (PART 1 & PART 2)

PART 1: COMBINATIONAL CIRCUITS (GROUP A)

Experiment 1: 4-to-1 Multiplexer

Verilog Design Code (Behavioral Modeling):

```
module mux4to1 (
    input wire [3:0] in,
    input wire [1:0] sel,
    output reg out
);
    always @(*) begin
        case (sel)
            2'b00: out = in[0];
            2'b01: out = in[1];
            2'b10: out = in[2];
            2'b11: out = in[3];
            default: out = 1'b0;
        endcase
    end
endmodule
```

Testbench Code:

```
`timescale 1ns/1ps
module mux4to1_tb;
    reg [3:0] in; reg [1:0] sel; wire out;
    mux4to1 uut (.in(in), .sel(sel), .out(out));
    initial begin
        in = 4'b1010; sel = 2'b00; #10;
        sel = 2'b01; #10;
        sel = 2'b10; #10;
        sel = 2'b11; #10;
        $finish;
    end
endmodule
```

Experiment 2: Full Adder using Two Half Adders

Verilog Design Code (Structural Hierarchy):

```
module half_adder (
    input wire A, B,
    output wire Sum, Carry
);
    assign Sum = A ^ B;
    assign Carry = A & B;
endmodule

module full_adder (
    input wire A, B, Cin,
    output wire Sum, Cout
);
    wire S1, C1, C2;
    half_adder ha1 (.A(A), .B(B), .Sum(S1), .Carry(C1));
    half_adder ha2 (.A(S1), .B(Cin), .Sum(Sum), .Carry(C2));
    assign Cout = C1 | C2;
endmodule
```

Full Adder Testbench Code:

```
`timescale 1ns/1ps
module full_adder_tb;
    reg A, B, Cin; wire Sum, Cout;
    full_adder uut (.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Cout(Cout));
    initial begin
        A=0; B=0; Cin=0; #10;
        A=0; B=1; Cin=1; #10;
        A=1; B=1; Cin=0; #10;
        A=1; B=1; Cin=1; #10;
        $finish;
    end
endmodule
```

Experiment 3: 4-Bit Arithmetic Logic Unit (ALU)

Verilog Design Code:

```
module alu_4bit (
    input wire [3:0] A, B,
    input wire [2:0] ALU_Sel,
    input wire Cin,
    output reg [3:0] Result,
    output reg Zero, Carry, Negative
);
    reg [4:0] temp;
    always @(*) begin
        case (ALU_Sel)
            3'b000: temp = A + B + Cin;     // ADD
            3'b001: temp = A - B;           // SUB
            3'b010: temp = A + 4'b0001;     // INC
            3'b011: temp = A - 4'b0001;     // DEC
            3'b100: temp = A & B;           // AND
            3'b101: temp = A | B;           // OR
            3'b110: temp = A ^ B;           // XOR
            3'b111: temp = ~A;              // NOT
            default: temp = 5'b0;
        endcase
        Result   = temp[3:0];
        Carry    = temp[4];
        Zero     = (Result == 4'b0000);
        Negative = Result[3];
    end
endmodule
```

Testbench Code:

```
`timescale 1ns/1ps
module alu_4bit_tb;
    reg [3:0] A, B; reg [2:0] ALU_Sel; reg Cin;
    wire [3:0] Result; wire Zero, Carry, Negative;
    alu_4bit uut (.A(A), .B(B), .ALU_Sel(ALU_Sel), .Cin(Cin),
                  .Result(Result), .Zero(Zero), .Carry(Carry), .Negative(Negative));
    initial begin
        A = 4'b0101; B = 4'b0011; Cin = 0; ALU_Sel = 3'b000; #10; // ADD
        ALU_Sel = 3'b001; #10; // SUB
        ALU_Sel = 3'b100; #10; // AND
        $finish;
    end
endmodule
```

Experiment 4: Binary-to-Gray Code Converter

Verilog Design Code:

```
module bin_to_gray (
    input wire [3:0] B,
    output wire [3:0] G
);
    assign G[3] = B[3];
    assign G[2] = B[3] ^ B[2];
    assign G[1] = B[2] ^ B[1];
    assign G[0] = B[1] ^ B[0];
endmodule
```

Testbench Code:

```
`timescale 1ns/1ps
module bin_to_gray_tb;
    reg [3:0] B; wire [3:0] G;
    bin_to_gray uut (.B(B), .G(G));
    initial begin
        B = 4'b0000; #10;
        B = 4'b0100; #10;
        B = 4'b1101; #10;
        B = 4'b1111; #10;
        $finish;
    end
endmodule
```

PART 2: SEQUENTIAL CIRCUITS (GROUP B)

Experiment 1: JK Flip-Flop & T Flip-Flop

JK Flip-Flop Design Code:

```
module jk_ff (
    input wire j, k, clk, rst,
    output reg Q,
    output wire Q_bar
);
    always @(posedge clk) begin
        if (rst)
            Q <= 1'b0;
        else begin
            case ({j, k})
                2'b00: Q <= Q;
                2'b01: Q <= 1'b0;
                2'b10: Q <= 1'b1;
                2'b11: Q <= ~Q;
            endcase
        end
    end
    assign Q_bar = ~Q;
endmodule
```

T Flip-Flop Design Code:

```
module t_flip_flop (
    input wire clk, reset, T,
    output reg Q
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            Q <= 1'b0;
        else if (T)
            Q <= ~Q;
        else
            Q <= Q;
    end
endmodule
```

JK Flip-Flop Testbench Code:

```
`timescale 1ns/1ps
module jk_ff_tb;
    reg j, k, clk, rst; wire Q, Q_bar;
    jk_ff uut (.j(j), .k(k), .clk(clk), .rst(rst), .Q(Q), .Q_bar(Q_bar));
    initial clk = 0; always #5 clk = ~clk;
    initial begin
        rst = 1; j = 0; k = 0; #12; rst = 0;
        j = 1; k = 0; #10; // Set
        j = 0; k = 1; #10; // Reset
        j = 1; k = 1; #20; // Toggle
        $finish;
    end
endmodule
```

Experiment 2: 3-Bit Asynchronous (Ripple) Counter

Verilog Design Code:

```
module async_counter_3bit (
    input wire clk, rst,
    output wire [2:0] q
);
    reg q0, q1, q2;

    always @(posedge clk or posedge rst) begin
        if (rst) q0 <= 1'b0;
        else     q0 <= ~q0;
    end

    always @(posedge q0 or posedge rst) begin
        if (rst) q1 <= 1'b0;
        else     q1 <= ~q1;
    end

    always @(posedge q1 or posedge rst) begin
        if (rst) q2 <= 1'b0;
        else     q2 <= ~q2;
    end

    assign q = {q2, q1, q0};
endmodule
```

Testbench Code:

```
`timescale 1ns/1ps
module async_counter_3bit_tb;
    reg clk, rst; wire [2:0] q;
    async_counter_3bit uut (.clk(clk), .rst(rst), .q(q));
    initial clk = 0; always #5 clk = ~clk;
    initial begin
        rst = 1; #12; rst = 0; #160;
        $finish;
    end
endmodule
```

Experiment 3: 3-Bit Synchronous Up/Down Counter

Verilog Design Code:

```
module sync_up_down_counter (
    input wire clk, rst, up_down,
    output reg [2:0] count
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 3'b000;
        else if (up_down)
            count <= count + 1'b1; // Up Count when up_down=1
        else
            count <= count - 1'b1; // Down Count when up_down=0
    end
endmodule
```

Testbench Code:

```
`timescale 1ns/1ps
module sync_up_down_counter_tb;
    reg clk, rst, up_down; wire [2:0] count;
    sync_up_down_counter uut (.clk(clk), .rst(rst), .up_down(up_down), .count(count));
    initial clk = 0; always #5 clk = ~clk;
    initial begin
        rst = 1; up_down = 1; #12; rst = 0;
        #80;          // Count Up
        up_down = 0;  // Switch to Down
        #80;
        $finish;
    end
endmodule
```

Experiment 4: Johnson Counter & Ring Counter

Ring Counter Design Code:

```
module ring_counter (
    input wire clk, rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0001; // Initial state
        else
            q <= {q[2:0], q[3]}; // Rotate left
    end
endmodule
```

Johnson Counter Design Code:

```
module johnson_counter (
    input wire clk, rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= {q[2:0], ~q[3]}; // Shift with inverted feedback
    end
endmodule
```

Ring Counter Testbench Code:

```
`timescale 1ns/1ps
module ring_counter_tb;
    reg clk, rst; wire [3:0] q;
    ring_counter uut (.clk(clk), .rst(rst), .q(q));
    initial clk = 0; always #5 clk = ~clk;
    initial begin
        rst = 1; #12; rst = 0; #100;
        $finish;
    end
endmodule
```

Experiment 5: Serial-In Serial-Out (SISO) Shift Register

Verilog Design Code:

```
module siso_shift_register (
    input wire clk, rst, s_in,
    output wire s_out
);
    reg [3:0] shift_reg;

    always @(posedge clk or posedge rst) begin
        if (rst)
            shift_reg <= 4'b0000;
        else
            shift_reg <= {shift_reg[2:0], s_in}; // Shift left & insert s_in
    end

    assign s_out = shift_reg[3];
endmodule
```

Testbench Code:

```
`timescale 1ns/1ps
module siso_shift_register_tb;
    reg clk, rst, s_in; wire s_out;
    siso_shift_register uut (.clk(clk), .rst(rst), .s_in(s_in), .s_out(s_out));
    initial clk = 0; always #5 clk = ~clk;
    initial begin
        rst = 1; s_in = 0; #12; rst = 0;
        s_in = 1; #10;
        s_in = 0; #10;
        s_in = 1; #10;
        s_in = 1; #50;
        $finish;
    end
endmodule
```